

MEDIATEK

MT8676 Android SDK User Manual

Version: 1.0
Release date: 2024-08-12

Use of this document and any information contained therein is subject to the terms and conditions set forth in Exhibit 1. This document is subject to change without notice.

版本记录

版本	日期	作者	描述
1.0	2024-08-12	胡丹凤	正式版

目录

版本记录 2

目录 3

图片目录 3

1 SDK 4

 1.1 概述 4

 1.1.1 简单介绍 4

 1.1.2 SDK 功能介绍 4

 1.1.3 注意事项 4

 1.2 架构/流程概述 4

 1.2.1 内部结构图 4

 1.2.2 API 列表 5

 1.3 配置/客制化指南 55

 1.3.1 主要功能介绍 55

 1.3.2 碰撞检测特别说明 57

 1.4 常见问题/故障排除 58

 1.4.1 SDK 常见问题 58

 1.4.2 调试日志开关方法 59

附件一 附加条款 60

图片目录

图 1-1. SDK 内部结构 5

图 1-2. SPM SDK 流程 5

1 SDK

1.1 概述

1.1.1 简单介绍

本章节主要介绍 MT8676 SDK 接口以及常见问题的处理方法。

SmartPlatform 是一个特殊的产品，MediaTek 为了降低客户 APK 的开发难度，实现更多定制化的需求，让开发者快速、方便地开发出好的应用，提供了 SmartPlatform 独有的一套 SDK 接口。

1.1.2 SDK 功能介绍

SDK 提供 API 给客户调用，每个摄像头可以独立进行 preview、capture、record、YUV callback (PictureSequence)。且支持多进程同时使用一个摄像头。

1.1.3 注意事项

- 考虑到同步和效率问题，SDK 中的回调是直接在线程中处理的，并未新开线程处理。所以应用程序在收到回调之后，最好开启一个新线程来处理；如果不另外开启新线程处理，请不要在回调中继续调用 SDK 接口（可能造成死锁）；也不要再在回调中执行耗时较长的任务。
- 关于拍照流程，为了避免拍照延长和 preview/record 中断，目前拍照和 preview 已进行了绑定。如果要调整拍照的大小，需要在调用 setPreviewSurface 之前通过 setPictureSize 进行设定。

1.2 架构/流程概述

1.2.1 内部结构图

MT8676 SDK 内部结构如图 1-1 所示：

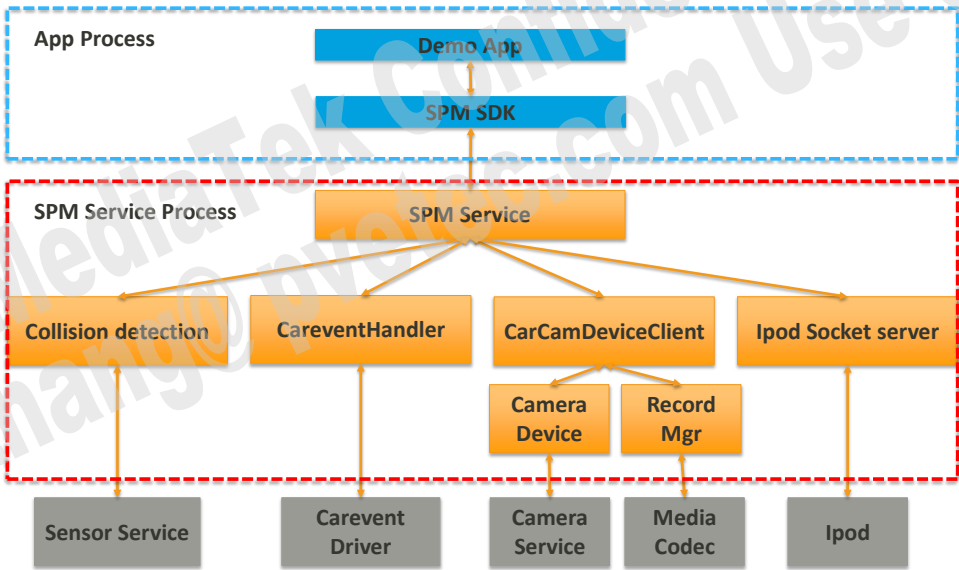


图1-1. SDK 内部结构

MT8676 SDK 流程如图 1-2 所示：

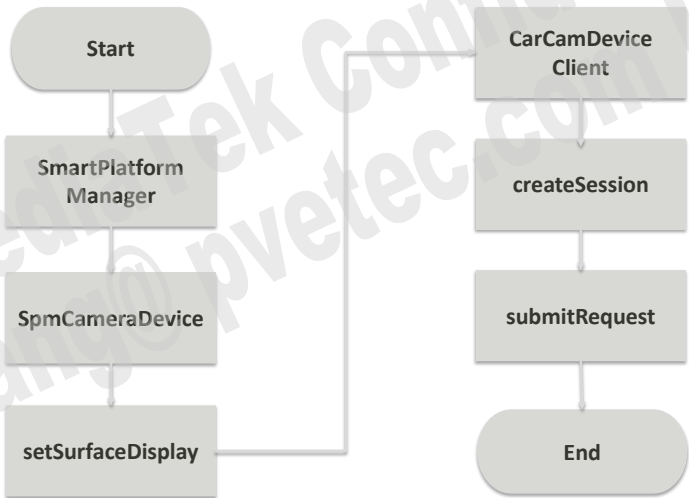


图1-2. SPM SDK 流程

1.2.2 API 列表

1.2.2.1 SmartPlatformManager

描述：

SmartPlatform SDK 的主要接口类，单例模式。管理平台的摄像头 录制，碰撞检测，关机状态设置等。

引入（导入） SmartPlatformManager：

```
import com.mediatek.smartplatform.SmartPlatformManager;
```

实例化(创建)对象:

```
SmartPlatformManager mSmartPlatformManager = SmartPlatformManager.get();
```

1.2.2.1.1 get()**函数:**

```
public static SmartPlatformManager get()
```

返回值:

返回一个 SmartPlatformManager 实例。

细节:

SmartPlatformManager 是以单例的模式存在应用程序中，这意味着同一个应用程序在任何地方调用此方法得到的 SmartPlatformManager 实例都是同一个。SmartPlatformManager 实例在该类被加载的时候就创建出来了，属于“饿汉式”。

示例:

```
SmartPlatformManager mSmartPlatformManager = SmartPlatformManager.get();
```

1.2.2.1.2 isServiceAlive**函数:**

```
public boolean isServiceAlive()
```

返回值:

APK 进程获取的 SmartPlatform service 和 Camera Service 是否可用。

细节:

- 如果 SmartPlatform Service 或者 Camera Service 还没有被注册到系统服务，或者注册失败，该接口会返回 false，这意味着调用 SmartPlatformManager 对象的一些方法会出现异常。所以应用程序需要先确保 SmartPlatform Service 可用后，再调用相关接口。
- 这个接口会返回 false 的情况一般发生在 smartplatformserver 进程异常退出后，smartplatformserver 进程会重启，此时应用程序也会重启。但是可能因为应用程序比 smartplatformserver 进程先重启，这时 Smartplatform Service 还没有就绪，所以应用程序此时调用相关接口就会发生空指针异常，导致应用程序退出。
- 针对上述的情况，建议针对应用程序做以下逻辑处理：
 - (1) 应用程序异常退出重启时，先判断 service 是否可用（只需要判断一次即可）。
 - (2) 如果不可用，需要延时等待 service 就绪。在 service 没有就绪之前，不要调用相关接口（比如 openCameraDevice()）。
 - (3) Service 就绪后，按照正常的流程往下进行。

示例:

```
SmartPlatformManager mSmartPlatformManager = SmartPlatformManager.get();
if (!mSmartPlatformManager.isServiceAlive()) {
    //可以延时等待后再去判断是否可用
    mHandler.sendMessageDelayed(INIT_DELAY, 500);
} else {
    //走正常的 初始化流程
}
```

1.2.2.1.3 getCameraIdList

函数:

```
public String[] getCameraIdList ()
```

返回值:

返回当前平台连接的摄像头设备的列表，包含正在使用的摄像头。

细节:

该函数返回的摄像头列表指的是平台当前实际连接到的摄像头。

与 android.hardware.camera2.CameraManager 中 getCameraIdList 不同，该函数会返回在位（1）、不在位（0）、正在使用（-2）的摄像头，CameraManager 只会返回在位的摄像头，具体状态信息参考 getCameraStatus。

示例:

```
SmartPlatformManager mSmartPlatformManager = SmartPlatformManager.get();
String cameraList[] = mSmartPlatformManager.getCameraIdList();
```

1.2.2.1.4 getAvmCameraId

函数:

```
public String getAvmCameraId ()
```

返回值:

返回当前平台连接的 AVM 摄像头设备的 ID。

细节:

该函数得到的摄像头列表指的是平台当前实际连接到的 AVM camera ID。这些 camera ID 主要用于 CarcorderDemo APK 调用，适用于 AVM DVR。

示例:

```
SmartPlatformManager mSmartPlatformManager = SmartPlatformManager.get();
String cameraId = mSmartPlatformManager.getAvmCameraId();
```

1.2.2.1.5 getCameraCharacteristics

函数:

```
public CameraCharacteristics getCameraCharacteristics (String cameraId)
```

参数:

cameraId 摄像头对应的标识

返回值:

数据类型为 android.hardware.camera2.CameraCharacteristics。cameraId 对应摄像头的设备功能。

细节:

cameraId 通过 getCameraIdList 获取此 API 与 android.hardware.camera2.CameraManager 中 getCameraCharacteristics 等同。

示例:

```
SmartPlatformManager mSmartPlatformManager = SmartPlatformManager.get();  
String cameraList[] = mSmartPlatformManager.getCameraIdList();  
mSmartPlatformManager.getCameraCharacteristics(cameraList[0]);
```

1.2.2.1.6 openCameraDevice

函数:

```
public SpmCameraDevice openCameraDevice (String cameraIdStr)
```

参数:

cameraIdStr 摄像头对应的标识

返回值:

返回一个 SpmCameraDevice 对象。

细节:

- 参数 cameraIdStr 是摄像头对应的标识 ID，每个摄像头的 ID 值都不同，这个 ID 通过 getCameraIdList 获取。
- 同一个应用程序内每次使用同一个 ID 调用该函数得到的 CameraDevice 对象是一样的。对象只会在第一次调用时被创建。

示例:

```
SmartPlatformManager mSmartPlatformManager = SmartPlatformManager.get();  
String ids[] = mSmartPlatformManager.getCameraIdList();  
SpmCameraDevice cameraDevice0= mSmartPlatformManager.openCameraDevice(ids[0]);
```

1.2.2.1.7 openAvmCameraDevice

函数:

```
public SpmCameraDevice openCameraDevice()
```

返回值:

返回一个 SpmCameraDevice 对象。

细节:

同一个应用程序内每次调用该函数得到的 CameraDevice 对象是一样的。对象只会在第一次调用时被创建。

示例:

```
SmartPlatformManager mSmartPlatformManager = SmartPlatformManager.get();  
SpmCameraDevice avmcameraDevice= mSmartPlatformManager.openAvmCameraDevice();
```

1.2.2.1.8 closeCameraDevice

函数:

```
public void closeCameraDevice(String cameraId)
```

参数:

cameraId 摄像头对应的标识

细节:

- 参数 cameraId 是摄像头对应的标识 ID，每个摄像头的 ID 值都不同，这个 ID 通过 getCameraIdList 获取。如果不需要使用此 CameraDevice，应该调用此函数把它关掉，以便资源的释放和回收。
- 调用了此函数之后，之前通过 openCameraDevice 得到的所有引用将不再可用。

示例:

```
SmartPlatformManager mSmartPlatformManager = SmartPlatformManager.get();  
String ids[] = mSmartPlatformManager.getCameraIdList();  
SpmCameraDevice cameraDevice0= mSmartPlatformManager.openCameraDevice(ids[0]);  
mSmartPlatformManager.closeCameraDevice(ids[0]);
```

1.2.2.1.9 closeAvmCameraDevice

函数:

```
public void closeAvmCameraDevice()
```

细节:

调用了此函数之后，之前通过 openAvmCameraDevice 得到的引用将不再可用。

示例:

```
SmartPlatformManager mSmartPlatformManager = SmartPlatformManager.get();  
SpmCameraDevice avmcameraDevice0= mSmartPlatformManager.openAvmCameraDevice();  
mSmartPlatformManager.closeAvmCameraDevice();
```

1.2.2.1.10 addServiceDeathCallback

函数:

```
public void addServiceDeathCallback(ServiceDeathCallback callback)
```

参数:

callback SmartPlatformServer 进程退出（死掉）时的回调

细节:

- ServiceDeathCallback 定义如下:

```
public interface ServiceDeathCallback {  
    void onDeath(int arg1,String arg2);  
}
```

- 同一个 ServiceDeathCallback 被多次添加，只会存在一次回调。
- 该回调被触发意味着 SmartPlatformServer 进程已经死掉了，此时 SmartPlatform Service 相关服务此时不再可用，目前的机制是 SmartPlatformServer 死掉（异常被杀死）后会重新加载运行。应用程序需要重新获取服务，因为之前 SmartPlatformManager, SpmCameraDevice 对象都无法继续使用了。

示例:

```
SmartPlatformManager mSmartPlatformManager = SmartPlatformManager.get();  
mSmartPlatformManager.addServiceDeathCallback (  
    new ServiceDeathCallback() {  
        public void onDeath(int arg1,String arg2){  
            }  
    }  
);
```

1.2.2.1.11 removeServiceDeathCallback

函数:

```
public void removeServiceDeathCallback (ServiceDeathCallback callback)
```

参数:

callback SmartPlatformServer 进程退出（死掉）时的回调

细节:

移除注册的回调对象

示例:

```
SmartPlatformManager mSmartPlatformManager = SmartPlatformManager.get();  
mSmartPlatformManager.removeServiceDeathCallback(callback);
```

1.2.2.1.12 addCameraAvailableCallback

函数:

```
public void addCameraAvailableCallback(ServiceDeathCallback callback)
```

参数:

callback 返回摄像头状态的回调

细节:

- CameraAvailableCallback 定义如下:

```
public interface CameraAvailableCallback {  
    void onAvailable (String cameraId, int status);  
}
```

- 同一个 CameraAvailableCallback 被多次添加，只会存在一次回调。
- 该回调被触发时意味着摄像头的新增或移除，被移除时可以做停止录制相关操作。

示例:

```
SmartPlatformManager mSmartPlatformManager = SmartPlatformManager.get();  
mSmartPlatformManager.addCameraAvailableCallback (  
    new CameraAvailableCallback () {  
        public void onAvailable (String cameraId, int status){  
            }  
    }  
);
```

1.2.2.1.13 removeCameraAvailableCallback

函数:

```
public void removeCameraAvailableCallback (CameraAvailableCallback callback)
```

参数:

callback 返回摄像头状态的回调

细节:

移除注册的回调对象

示例:

```
SmartPlatformManager mSmartPlatformManager = SmartPlatformManager.get();  
mSmartPlatformManager.removeCameraAvailableCallback(callback);
```

1.2.2.1.14 getIpodProxy

函数:

```
public IpodProxy getIpodProxy()
```

返回值:

返回一个 IpodProxy 对象。

细节:

参考 IpodProxy 说明。

示例:

```
SmartPlatformManager mSmartPlatformManager = SmartPlatformManager.get();
IpodProxy mIpodProxy=mSmartPlatformManager.getIpodProxy();
```

1.2.2.1.15 getCollisionProxy

函数:

```
public CollisionProxy getCollisionProxy()
```

返回值:

返回一个 CollisionProxy 对象。

细节:

参考 CollisionProxy 说明。

示例:

```
SmartPlatformManager mSmartPlatformManager = SmartPlatformManager.get();
CollisionProxy mCollisionProxy=mSmartPlatformManager.getCollisionProxy();
```

1.2.2.1.16 getCarEventProxy

函数:

```
public CarEventProxy getCarEventProxy()
```

返回值:

返回一个 CarEventProxy 对象。

细节:

参考 CarEventProxy 说明。

示例:

```
SmartPlatformManager mSmartPlatformManager = SmartPlatformManager.get();
CarEventProxy mCarEventProxy=mSmartPlatformManager.getCarEventProxy();
```

1.2.2.1.17 getNumberOfCameras

函数:

```
public int getNumberOfCamera()
```

返回值:

返回可用的摄像头个数，包含在位、不在位、正在使用的摄像头。

细节:

参考 `getCameraIdList` 说明，实质上该方法返回的是 `getCameraIdList` 返回的数组大小。

示例:

```
SmartPlatformManager mSmartPlatformManager = SmartPlatformManager.get();  
int cameraNumber = mSmartPlatformManager.getNumberOfCamera();
```

1.2.2.1.18 getCameraStatus

函数:

```
public int getCameraStatus(String cameraId)
```

返回值:

返回所要获取的 `cameraId` 的状态。

细节:

状态含义如 `frameworks/av/camera/aidl/android/hardware/ICameraServiceListener.aidl`:

```
// Device physically unplugged  
const int STATUS_NOT_PRESENT = 0; // 不在位  
// Device physically has been plugged in and the camera can be used exclusively  
const int STATUS_PRESENT = 1; // 在位可用  
// Device physically has been plugged in but it will not be connect-able until enumeration  
is complete  
const int STATUS_ENUMERATING = 2; // enumator provider 运行时 camera 返回的状态，enumator 完成后  
会转到 PRESENT 状态  
// Camera is in use by another app and cannot be used exclusively  
const int STATUS_NOT_AVAILABLE = -2; // 其他应用程序进程在使用，由于 MediaTek SDK 支持多进程使用，所  
以是可用的  
// Use to initialize variables only  
const int STATUS_UNKNOWN = -1; // 初始状态
```

示例:

```
SmartPlatformManager mSmartPlatformManager = SmartPlatformManager.get();  
int cameraStatus = mSmartPlatformManager.getCameraStatus(cameraId);
```

1.2.2.2 PreviewSource

引入（导入）`PreviewSource`:

```
import com.mediatek.smartplatform.PreviewSource;
```

对象说明:

用于区分多个预览（preview），同一个摄像头设备可以进行多个预览操作。通常使用 `GENERAL_CAMERA` 或者 `PREVIEW_[0-4]` 进行区分。

1.2.2.3 RecordSource

引入（导入）RecordSource:

```
import com.mediatek.smartplatform.RecordSource;
```

对象说明:

用于区分录制（Record）的类型，同一个摄像头设备可以进行多路不同分辨率的录制。通常使用 `GENERAL_CAMERA` 或者 `RECORD_[0-4]` 进行录制。如果是低分辨率录制，可以选择 `GENERAL_CAMERA_SUB` 或者 `RECORD_EX_[0-4]`。

1.2.2.4 PictureSequenceSource

引入（导入）PictureSequenceSource:

```
import com.mediatek.smartplatform.PictureSequenceSource;
```

对象说明:

用于区分 `PictureSequenceSource` 的类型，同一个摄像头设备可以获取多路不同分辨率的连续图片，通常使用 `GENERAL_CAMERA` 获取。

1.2.2.5 RecordConfiguration

引入（导入）RecordConfiguration:

```
import com.mediatek.smartplatform.RecordConfiguration;
```

实例化(创建)对象:

```
RecordConfiguration recordConfig= RecordConfiguration.get(recordSource);
```

1.2.2.5.1 mCamcorderProfile

类别: `android.media.CamcorderProfile`

细节: 录像视频文件的格式，帧率，视频画面大小等设定。

1.2.2.5.2 mOutPutFilePath

类别: java.lang.String

细节: 录像视频文件的保存路径

1.2.2.5.3 mOutPutFileName

类别: java.lang.String

细节: 录像视频文件的前缀

1.2.2.5.4 mVideoCallback

类别: com.mediatek.smartplatform.SpmCameraDevice.VideoCallback

细节: 录像视频相关事件发生时的回调

- VideoCallback 定义如下:

```
public interface VideoCallback {  
    public static final int VIDEO_EVNET_ADD_FILE_IN_GALLERY=0;  
    public static final int VIDEO_EVNET_DELETE_FILE_IN_GALLERY=1;  
    public static final int VIDEO_EVNET_SDCARD_FULL=2;  
    public static final int VIDEO_EVNET_RECORD_STATUS_START=3;  
    public static final int VIDEO_EVNET_RECORD_STATUS_STOP =4;  
    public static final int VIDEO_EVNET_RECORD_RECORDING_ERROR=5;  
    public static final int VIDEO_EVNET_RECORD_SDCARD_DAMAGED=6;  
    public static final int VIDEO_EVNET_LOWRES_KEYPOINT_START =7;  
    public static final int VIDEO_EVNET_LOWRES_KEYPOINT_STOP =8;  
    public static final int VIDEO_EVNET_KEYPOINT_START =9;  
    public static final int VIDEO_EVNET_KEYPOINT_STOP =10;  
  
    void onVideoTaken(int eventType, int cameraId, int recorderSource, String videoname, int  
        startTime, int endTime);  
    void onVideoTake(VideoInfo videoInfo);  
    void onVideoFrame(byte[] data,int dataType, int size, int cameraId, int recorderSource);  
}
```

- VideoInfo 是 eventType, cameraId 等等的集合, 后续不再使用多参数的 onVideoTaken。
- onVideoTaken 中[VideoInfo] eventType 含义如下:
 - VIDEO_EVNET_ADD_FILE_IN_GALLERY 录像完成, 并且视频文件被成功保存, 通过 videoname 可以得到视频文件的路径。
 - VIDEO_EVNET_DELETE_FILE_IN_GALLERY 视频文件被删除时, 会收到此通知, 一般发生在循环录制时, 通过删除之前的视频文件释放空间。
 - VIDEO_EVNET_SDCARD_FULL 录制视频的时候检测到 SD Card 已经满了。

- `VIDEO_EVNET_RECORD_RECORDING_ERROR` 录制视频过程中发生了错误。
- `VIDEO_EVNET_RECORD_SDCARD_DAMAGED` 录制视频时检测到 SD Card 被损坏了，如果 SD Card 的写入速度小于某个阈值，也可能被认为损坏了。
- `VIDEO_EVNET_LOWRES_KEYPOINT_START` 低分辨率打点开始。
- `VIDEO_EVNET_LOWRES_KEYPOINT_STOP` 低分辨率打点结束。
- `VIDEO_EVNET_KEYPOINT_START` 打点开始。
- `VIDEO_EVNET_KEYPOINT_STOP` 打点结束。
- `onVideoTaken` 中 `[VideoInfo] startTime, int endTime`
当收到 `VIDEO_EVNET_ADD_FILE_IN_GALLERY` 的时候，表示视频文件录制的开始时间和结束时间。注意需设置 `enableVideoWithTimeCallback(true)`。
- `onVideoFrame`: 通过此接口可以接收每一帧视频数据。
 - `data`: 保存数据的缓冲区
 - `dataType`: 保留参数，可能被用来区分是高清还是标清数据。
 - `size`: 表示视频数据的大小，一般情况下小于 `data` 数组的大小，每次回调得到的视频数据大小可能不一样。
 - `cameraId`: 返回当前视频数据的 `cameraId`。
 - `recorderType`: 返回当前视频数据的 `recorderType`。
- `recorderSource` 与 `startRecord` 中的 `recorderSource` 相对应。

1.2.2.5.5 mAudioCallback

类别: `com.mediatek.smartplatform.SpmCameraDevice.AudioCallback`

细节: 音频数据的回调

- `AudioCallback` 定义如下:

```
public interface AudioCallback {
    void onAudioFrame(byte[] data, int dataType, int size, int cameraId, int recorderSource);
}
```

- `onAudioFrame`: 通过此接口可以接收每一帧音频数据。
 - `data`: 保存数据的缓冲区。
 - `dataType`: 保留参数。
 - `size`: 有效数据的大小，一般情况下小于 `data` 的大小。
 - `cameraId`: 当前返回音频数据的 `cameraId`。
- `recorderSource`: 当前返回音频数据的 `recorderSource` 与 `startRecord` 中的 `recorderSource` 相对应。

1.2.2.5.6 mRecordStatusCallback

类别: com.mediatek.smartplatform.SpmCameraDevice.RecordStatusCallback

细节: 录像状态发生改变的回调

- RecordStatusCallbac 定义如下:

```
public interface RecordStatusCallback{
    public static fianl int RECORD_STATUS_START      = 0;
    public static fianl int RECORD_STATUS_STOP      = 1;
    public static fianl int RECORD_STATUS_RECORDING  = 2;
    void onRecordStatusChanged(int status,int cameraid,int recorderSource);
}
```

- 当录像状态发生变化时会回调。

1.2.2.5.7 mKeypointCallback

类别: com.mediatek.smartplatform.SpmCameraDevice.KeyPointCallback

细节: 打点数据通知到应用程序时的回调

- KeypointCallback 定义如下:

```
public interface KeypointCallback{
void onKeypointFrame(byte[] data,int dataType,int size,String cameraId, int recorderSource);
}
```

- onKeypointFrame: 通过此接口接收已编码的数据。
 - data: 保存数据的缓冲区, 每次回调 onKeypointFrame 时, 使用的都是同一个缓冲区。如果想要保存数据, 需要重新拷贝, 不然下一次会被覆盖。
 - dataType: 保留参数, 可能被用来区分是高清还是标清数据。
 - size: 标示打点 (keypoint) 数据的大小, 一般情况下小于 data 的大小, 这个数据大小一般是 128KB, 包含视频和音频数据, 格式为 ts。

1.2.2.5.8 mVideoFrameMode

类别: int

细节: 标识要怎么处理视频数据。

VideoFrameMode 的定义如下:

- VIDEO_FRAME_MODE_DISABLE (0), //录制到文件
- VIDEO_FRAME_MODE_SOURCE (1), //以 h264 格式提供给应用程序
- VIDEO_FRAME_MODE_PACKET (2), //以 ts 格式提供给应用程序

- `VIDEO_FRAME_MODE_DUAL` (3); //以 ts 格式录制到文件,同时提供给应用程序

1.2.2.5.9 mAudioSource

类别: `int`

细节: 标识音频的来源。参考 `android.media.MediaRecorder.AudioSource`, 一般为 `CAMCORDER`。

1.2.2.5.10 mKeypointSpanLimit

类别: `int`

细节: 最大的时间跨度, 单位是秒。

- 如果当前打点视频文件的时间和前一个视频文件的时间超过设定的最大时间跨度, 就不再截取前一个视频文件作为打点保护视频。
- 此参数需要在 `startRecord` 之前设置, 参数是一个整数值, 单位是秒。
- 如果应用程序不设置此参数, 则默认的跨度时间是当前设置的录制文件时间长度。

1.2.2.5.11 mRecordingMuteAudio

类别: `boolean`

细节: 是否关闭麦克风

参数为 `true` 表示关闭麦克风, 这样录出来的视频就没有声音了, 参数为 `false` 表示打开。

1.2.2.5.12 mVideoCbFrameRate

类别: `int`

细节: 设置视频回调的帧率。

1.2.2.5.13 mLockFilePath

类别: `String`

细节: 保护视频文件的保存目录。

设置受保护的视频文件的保存目录, 默认的路径是 `/sdcard/DCIM/camera/protect`。

1.2.2.5.14 mVideoRotateSize

类别: `int`

细节: 录制文件的最大 size , 以字节 (bytes) 为单位计算。当录制文件达到 size, 会存放在新的文件中。

1.2.2.5.15 mVideoRotateDuration

类别: `int`

细节: 设置保存的每个视频文件的时间长度, 单位是毫秒。

1.2.2.5.16 mEnableRecordStartRing

类别: `boolean`

细节: 开关录制铃声

1.2.2.5.17 mVideoBitRateMin

类别: `int`

细节: 视频录制最小的比特率

设置视频文件的比特率范围, 在录像的时候如果写文件的速度变慢, 底层会降低比特率; 如果写得快, 就会调高比特率。降低比特率是为了避免写文件太慢而导致缓冲区积压。即使写文件速度再慢, 调整后的比特率也不会小于设置的最小值, 同样提高的比特率也不会超过最大值。

1.2.2.5.18 mVideoBitRateMax

类别: `int`

细节: 视频录制最大的比特率

设置视频文件的比特率范围, 在录像的时候如果写文件的速度变慢, 底层会降低比特率; 如果写得快, 就会调高比特率。降低比特率是为了避免写文件太慢而导致缓冲区积压。即使写文件速度再慢, 调整后的比特率也不会小于设置的最小值, 同样提高的比特率也不会超过最大值。

1.2.2.5.19 mVideoCycleDeleteFileNum

类别: int

细节: 循环删除的文件数量, 当存储空间不够时删除的文件数量。

1.2.2.5.20 mReduceRecordingFps

类别: int

细节: 降低视频录制的帧率百分比。

FULL: 100%

HALF: 50%

THIRTY_PERCENT: 30%

QUARTER: 25%

1.2.2.6 PictureConfiguration

引入 (导入) PictureConfiguration:

```
import com.mediatek.smartplatform.PictureConfiguration;
```

实例化(创建)对象:

```
PictureConfiguration recordConfig= PictureConfiguration.get(recordSource);
```

1.2.2.6.1 mImageCallback

建议直接使用 ImageDataCallback。

类别: com.mediatek.smartplatform.ImageReaderEx.ImageCallback;

细节: 需要获取的图像的 YUV data 的回调。

```
ImageCallback {
int IMAGE_FORMAT_YUV_420_888 = ImageFormat.YUV_420_888;
int IMAGE_FORMAT_JPEG =ImageFormat.JPEG
int IMAGE_FORMAT_NV21 =ImageFormat.NV21
int IMAGE_DATA_RAW =0;
int IMAGE_DATE_FILE =1;
void onImageAvailable(String cameraId, int format, int status , byte[] data, String path)
}
```

imageFormat: 目前只支持 YUV_420_888 和 JPEG, NV21。

Datatype 可以为 file 或者 raw。 为 raw 会通过 callback 返回; 为 file 则保存在 path 目录。 保存为 file 一般用来进行测试。

1.2.2.6.2 mImageDataCallback

类别: `com.mediatek.smartplatform.SpmCameraDevice.ImageDataCallback`;

细节: 需要获取目标图像的 YUV DirectByteBuffer 的回调。

```
ImageDataCallback {  
    int IMAGE_FORMAT_YUV_420_888 = ImageFormat.YUV_420_888;  
    int IMAGE_FORMAT_JPEG = ImageFormat.JPEG;  
    int IMAGE_FORMAT_NV21 = ImageFormat.NV21;  
    int IMAGE_FORMAT_YV12 = ImageFormat.YV12;  
    int IMAGE_FORMAT_RGB_888 = PixelFormat.RGB_888;  
    int IMAGE_FORMAT_YUY2 = ImageFormat.YUY2;  
  
    int IMAGE_DATA_RAW = 0;  
    int IMAGE_DATA_FILE = 1;  
    int IMAGE_DATA_BUFFER = 2;  
    int IMAGE_DATA_IMAGE = 3;  
    void onImageAvailable(ImageDataCallbackInfo imageDataCallbackInfo);  
}
```

imageFormat: 目前支持 YUV_420_888 和 JPEG, NV21, YV12, RGB_888, YUY2 格式。

DataType 可以为 RAW, FILE, BUFFER 或者 IMAGE。

RAW: 将数据放在一个 `byte[]` 中回调回来。

FILE: 保存在平台, 会删除旧文件, 用于本地测试。

BUFFER: 将数据放在一个 `DirectByteBuffer` 中返回给应用程序, 减少复制次数。

IMAGE: 将数据放在一个图像中返回给应用程序。

1.2.2.6.3 mPath

类别: `String`

细节: 需要获取的图像的保存路径

此参数目前没有使用。

1.2.2.6.4 mImageFormat

类别: `int`

细节: 所注册回调需要获取的图像格式, 设置值为 `ImageCallback` 或 `mImageDataCallback` 中的 `format` 值, 标识了回调获取的图像类型。

1.2.2.6.5 mDateType

类别: int

细节: 需要获取的图像的 data 类型, 是 RAW, FILE, BUFFER, IMAGE 中的一种。

1.2.2.6.6 mPicWidth

类别: int

细节: 需要获取的图像的宽度

1.2.2.6.7 mPicHeight

类别: int

细节: 需要获取的图像的高度

1.2.2.6.8 mJpegHwEnc

类别: Boolean

细节: 是否使用硬件编码器将摄像头数据编码成 JPEG 格式, 目前没有使用这个标志。

1.2.2.7 SpmCameraDevice

引入 (导入) SpmCameraDevice:

```
import com.mediatek.smartplatform.SpmCameraDevice;
```

实例化(创建)对象:

```
SpmCameraDevice mCameraDevice =SmartPlatformManager.get().openCameraDevice("0");
```

1.2.2.7.1 getState

函数:

```
public int getState()
```

返回值:

返回当前摄像头 (Cameradevice) 所处的状态。

细节:

1. 摄像头的状态

STATE_IDLE, STATE_PREVIEW_[0-4], STATE_RECOR_[0-4], ...

2. 状态的说明

各种状态是“或”的关系，比如返回 0x21，则说明此 CameraDevice 正处于 STATE_PREVIEW_0 和 STATE_RECOR_0 状态。

3. 该接口返回的是底层保存的状态。如果应用程序 1 在预览，应用程序 2 在录制；若应用程序 1 异常退出，待应用程序 1 重新启动后，通过此接口获取的摄像头的当前状态依然是正在录制。

示例：

```
SpmCameraDevice mCameraDevice = SmartPlatformManager.get().openCameraDevice(ids[0]);
int state=mCameraDevice.getState();
```

1.2.2.7.2 getCameraCharacteristics

函数：

public CameraCharacteristics getCameraCharacteristics()

返回值：

数据类型为 android.hardware.camera2.CameraCharacteristics。当前摄像头的设备功能。

细节：

cameraId 通过 getCameraIdList 获取。
与 android.hardware.camera2.CameraManager 中 getCameraCharacteristics 和 SmartPlatformManager 中 getCameraCharacteristics 类似。通过此方法，可以获取摄像头的所有配置。
getParameters 中获取的参数也是通过 CameraCharacteristics 转换。

示例：

```
SpmCameraDevice mCameraDevice = SmartPlatformManager.get().openCameraDevice("0");
mCameraDevice.getCameraCharacteristics();
```

1.2.2.7.3 getParameters

函数：

public SpmParameters getParameters()

返回值：

SpmParameters 对象，SpmCameraDevice 参数的集合。

细节：

SpmCameraDevice 的参数信息可以通过 SpmParameters 得到并修改。

SpmParameters 中的参数是通过 CameraCharacteristics 转换得到的。目前可能存在某些参数未转换，若需要则可再增加转换。

示例：

```
SpmCameraDevice mCameraDevice = SmartPlatformManager.get().openCameraDevice("0");
SpmParameters params=mCameraDevice.getParameters();
```

1.2.2.7.4 setParameters

函数：

```
public void setParameters(SpmParameters params)
```

参数：

params 需要设置的参数集合

细节：

当要修改 SpmCameraDevice 一些参数时，必须要调用这个接口才能生效。

示例：

```
SpmCameraDevice mCameraDevice = SmartPlatformManager.get().openCameraDevice("0");
SpmParameters params=mCameraDevice.getParameters(); //先获取 params
mCameraDevice.setParameters(params); //修改之后再设置下去
```

1.2.2.7.5 setADASCallback

函数：

```
public void setADASCallback(ADASCallback callback)
```

参数：

callback ADAS 的回调

细节：

ADASCallback 定义如下：

```
public interface ADASCallback{
    void onADASCallback(ADASInfo info);
}
```

示例：

```
SpmCameraDevice mCameraDevice = SmartPlatformManager.get().openCameraDevice("0");
mCameraDevice.setADASCallback(
    new setADASCallback(){
        public void onADASCallback(ADASInfo info){
            //TODO
        }
    }
);
```

);

1.2.2.7.6 setPictureSize

函数:

```
public int setPictureSize(android.util.Size pictureSize)
```

参数:

pictureSize 图片大小

返回值:

返回 0 表示设置成功

示例:

```
SpmCameraDevice mDevice = SmartPlatformManager.get().openCameraDevice("0");
mDevice.setPictureSize (1280*720);
```

1.2.2.7.7 setPreviewSurface

函数:

```
public int setPreviewSurface(Surface surface, @PreviewSource.Format int previewSource)
```

参数:

surface 显示缓冲区的产生端

previewSource 预览源

返回值:

返回一个 boolean，表示设置是否成功。

细节:

参考 frameworks\base\core\java\android\view\Surface.java

示例:

```
SpmCameraDevice mDevice = SmartPlatformManager.get().openCameraDevice("0");
SurfaceView surfaceView=findViewById(R.id.surface_view);
SurfaceHolder holder=surfaceView.getHolder();
mDevice.setPreviewSurface(holder.getSurface(), PreviewSource.GENERAL_CAMERA);
```

1.2.2.7.8 startPreview

函数:

```
public void startPreview()
```

细节:

显示摄像头预览画面，在调用此 API 前，必须先调用 setPreviewSurface()。

示例：

```
SpmCameraDevice mDevice = SmartPlatformManager.get().openCameraDevice("");
SurfaceView surfaceView=findViewById(R.id.surface_view);
SurfaceHolder holder=surfaceView.getHolder();
mDevice.setPreviewSurface(holder.getSurface(), PreviewSource.GENERAL_CAMERA);
mDevice.startPreview();
```

1.2.2.7.9 stopPreview

函数：

```
public void stopPreview()
```

细节：

停止显示摄像头预览画面。停止预览后需要调用 setPreviewSurface null。

示例：

```
SpmCameraDevice mDevice = SmartPlatformManager.get().openCameraDevice("");
mDevice.stopPreview();
```

1.2.2.7.10 startADAS

函数：

```
public void startADAS()
```

细节：

- 打开 ADAS 功能。
- 需要 setADASCallback 来接收 ADAS 的相关数据。

示例：

```
SpmCameraDevice mDevice = SmartPlatformManager.get().openCameraDevice("");
mDevice.startADAS();
```

1.2.2.7.11 stopADAS

函数：

```
public void stopADAS()
```

细节：

关闭 ADAS 功能。

示例：

```
SpmCameraDevice mDevice = SmartPlatformManager.get().openCameraDevice("");
mDevice.stopADAS();
```

1.2.2.7.12 startRecord

函数:

```
public int startRecord(@RecordSource.Format int recordSource, RecordConfiguration recordConfig)
```

参数:

recordSource 录制的源

recordConfig 录制的设置

返回值:

返回一个整数值，表示启动录制是否成功，0 表示成功，否则失败。

细节:

参数设置见 RecordConfiguration

示例:

```
RecordConfiguration recordConfig= RecordConfiguration.get(recordSource);  
recordConfig.setVideoCallback(mVideoCallback);  
mCameraDevice.startRecord(RecordSource. GENERAL_CAMERA, recordConfig);
```

1.2.2.7.13 stopRecord

函数:

```
public void stopRecord(@RecordSource.Format int recordSource)
```

参数:

recordSource 录制的源

细节:

停止录制视频，和 startRecord()对应。

示例:

```
SpmCameraDevice Device = SmartPlatformManager.get().openCameraDevice("");  
mDevice.stopRecord();
```

1.2.2.7.14 lockRecordingVideo

函数:

```
public void lockRecordingVideo(int duration,String protectedType, @RecordSource.Format int recordSource)
```

参数:

duration 打点保护的时间长度

protectedType 打点保护类型, 为空串""或者"LowRes"

recordSource 录制的源

细节:

设置打点保护的时间长度时, 系统会将当前时间点前后各 duration 长度的视频文件保护起来, 所以打点保护的视频文件总长度为 2*duration 的时间长度。

第二个参数必须传入"LowRes", 底层会将打点视频文件转成 480P 并通知应用程序使用, 应用程序需要通过 KeypointCallback 来接收数据。

示例:

```
SpmCameraDevice mDevice = SmartPlatformManager.get().openCameraDevice("");  
mDevice.lockRecordingVideo(20, "LowRes", RecordSource.GENERAL_CAMERA);
```

1.2.2.7.15 unlockRecordingVideo

函数:

```
public void unlockRecordingVideo(String filename, @RecordSource.Format int recordSource)
```

参数:

filename 受保护的文件路径

recordSource 录制的源

细节:

将某个受保护 (protect) 目录下的视频文件移到普通的目录下。

示例:

```
SpmCameraDevice mDevice = SmartPlatformManager.get().openCameraDevice("");  
mDevice.unlockRecordingVideo(videofile, RecordSource.GENERAL_CAMERA);
```

1.2.2.7.16 setVideoRotateDuration

函数:

```
public void setVideoRotateDuration(int duration_ms, @RecordSource.Format int recordSource)
```

参数:

duration_ms 每个视频文件的时间长度, 单位是毫秒

recordSource 录制的源

细节:

设置保存的每个视频文件的时间长度, 需要在启动录像之后设置。如果需要在录像启动之前设置, 需要通过 RecordConfiguration 的 setVideoRotateDuration 来实现。设置的参数会在下一个录制的视频文件生效。

示例:

```
CameraDevice mDevice = SmartPlatformManager.get().openCameraDevice("");  
mDevice.setVideoRotateDuration(60*1000, RecordSource.GENERAL_CAMERA);
```

1.2.2.7.17 setVideoBitrateDyn**函数:**

```
public void setVideoBitrateDyn(int bitrate, int bAdjust, @RecordSource.Format int  
recordSource)
```

参数:

bitrate 视频比特率的值

bAdjust 保留参数, 暂未使用

recordSource 录制的源

细节:

动态调整录制的码率, 可以在录制过程中设置。

示例:

```
SpmCameraDevice mDevice = SmartPlatformManager.get().openCameraDevice("");  
mCameraDevice.setVideoBitrateDyn(6*1000*1000, 0, RecordSource.GENERAL_CAMERA);
```

1.2.2.7.18 setRecordingMuteAudio**函数:**

```
public void setRecordingMuteAudio(boolean isMuteAudio, @RecordSource.Format int  
recordSource)
```

参数:

isMuteAudio 是否关闭麦克风

recordSource 录制的源

细节:

参数为 true 表示关闭麦克风, 这样录出来的视频就没有声音了, 参数为 false 表示打开。

示例:

```
SpmCameraDevice mDevice = SmartPlatformManager.get().openCameraDevice("");  
mDevice.setRecordingMuteAudio(false, RecordSource.GENERAL_CAMERA);
```

1.2.2.7.19 setProtectRecording**函数:**

```
public void setProtectRecording(boolean ismotiondetect,boolean  
isrecordingstatus,int duration_ms, @RecordSource.Format int recordSource)
```

参数:

ismotiondetect 表示是否进行碰撞保护

isrecordingstatus 录制过程中碰撞, 还是非录制状态下碰撞

duration_ms 碰撞保护的时间

recordSource 录制的源

细节:

该接口用于启动和关闭碰撞保护, 第一个参数表示启动还是停止。

示例:

```
CameraDevice mCameraDevice = SmartPlatformManager.get().openCameraDevice(0);  
mCameraDevice.setProtectRecording(true, true, 5*60*1000, RecordSource.GENERAL_CAMERA)
```

1.2.2.7.20 flushCurRecFile

函数:

```
public void flushCurRecFile(@RecordSource.Format int recordSource)
```

参数:

recordSource 录制的源

细节:

Flush 当前录制文件, 将缓存数据写入到磁盘中。

示例:

```
SpmCameraDevice mDevice = SmartPlatformManager.get().openCameraDevice("\0");  
mDevice.flushCurRecFile (RecordSource.GENERAL_CAMERA);
```

1.2.2.7.21 setLockFilePath

函数:

```
public void setLockFilePath(String path, @RecordSource.Format int recordSource)
```

参数:

path 保存视频文件的目录

recordSource 录制的源

细节:

设定打点视频文件的目录, 默认的路径是/sdcard/DCIM/camera/protect/。该接口可在录制过程中打点之前进行调用。

示例:

```
SpmCameraDevice mDevice = SmartPlatformManager.get().openCameraDevice("");  
mDevice.setLockFilePath("/sdcard/DCIM/camera/video", RecordSource.GENERAL_CAMERA);
```

1.2.2.7.22 setRecordingSdcardPath**函数:**

```
public void setRecordingSdcardPath(String path, @RecordSource.Format int recordSource)
```

参数:

path 保存视频文件的目录

recordSource 录制的源

细节:

保存视频文件的目录，默认的路径是/sdcard/DCIM/camera。该接口可在录制过程中调用。

示例:

```
SpmCameraDevice mDevice = SmartPlatformManager.get().openCameraDevice("");  
mDevice.setRecordingSdcardPath("/sdcard/DCIM/camera/video", RecordSource.GENERAL_CAMERA);
```

1.2.2.7.23 enableShutterSound**函数:**

```
public void enableShutterSound(boolean shutter)
```

参数:

shutter 是否打开快门声音

细节:

参数为 true 表示打开快门声音，false 表示关闭。

示例:

```
SpmCameraDevice mDevice = SmartPlatformManager.get().openCameraDevice("");  
mDevice.enableShutterSound(false);
```

1.2.2.7.24 enableRecordSound**函数:**

```
public void enableRecordSound(boolean enable)
```

参数:

Enable 是否打开录制的提示声音

细节:

参数为 true 表示打开录制声音, false 表示关闭。

示例:

```
SpmCameraDevice mDevice = SmartPlatformManager.get().openCameraDevice("0");
mDevice.enableRecordSound(false);
```

1.2.2.7.25 setPictureSize

函数:

```
public void setPictureSize(android.util.Size PictureSize)
```

参数:

PictureSize 拍照图片的宽度和高度

细节:

用于设置拍照图片的大小, 需要在 setPreviewSurface 前进行设置。

1.2.2.7.26 takePicture

函数:

```
public void takePicture(String fileName,ShutterCallback shutter, CamPictureCallback jpeg)
```

参数:

fileName: 拍照图片的保存名字 (包含目录)

shutter: 快门提示音响起时的回调 (暂时无效)

jpeg: 拍照过程中的回调

细节:

- ShutterCallback 定义如下:

```
public interface ShutterCallback{
    void onShutter();
}
```

- CamPictureCallback 定义如下:

```
public interface CamPictureCallback{
    public static final int PICTURE_TAKEN_FALL = -1;//拍照失败
    public static final int PICTURE_TAKEN_SAVING = 0;//正在存储
    public static final int PICTURE_TAKEN_SUCCESS = 1;//拍照成功
    void onPictureTaken(int status,String cameraId,String fileName);
}
```

- onPictureTaken 中的 status 为拍照的状态：拍照失败，正在储存，拍照成功。cameraId 为摄像头标识，fileName 为拍照图片的保存名字。如果拍照失败，路径为空。此函数在拍照完成前、完成后，成功或失败都会被回调一次。

示例：

```
SpmCameraDevice mCameraDevice = SmartPlatformManager.get().openCameraDevice("0");
String fileName="/sdcard/DCIM/photo/"+filename;
mCameraDevice.takePicture(fileName, shutter, jpeg);
```

1.2.2.7.27 release

函数：

```
public boolean release()
```

细节：

注销该摄像头相关的 callback 和释放 listener。在 APK 退出时可调用此 API 来快速注销 callback 和释放 listener 以释放资源。

Return：

移除 listener 成功则返回 true，否则返回 false。

示例：

```
SpmCameraDevice mCameraDevice = SmartPlatformManager.get().openCameraDevice("0");
boolean isRemoveListener = mCameraDevice.release();
```

1.2.2.7.28 getCameraId

函数：

```
public String getCameraId()
```

返回值：

返回当前摄像头（SpmCameradevice）的 ID 值。

细节：

获取当前 SpmCameraDevice 对应的 camera ID。

示例：

```
SpmCameraDevice mDevice = SmartPlatformManager.get().openCameraDevice("0");
String id=mDevice.getCameraId();
```

1.2.2.7.29 startPictureSequence

函数：

```
public void startPictureSequence(@PictureSequenceSource int source PictureConfiguration config)
```

参数:

source: 获取连续图片的源

config: 获取连续图片的配置

细节:

此 API 用来连续获取图片。其中 YUV data 可以用于 ADAS 或者其它算法分析。如果 image size 为 null, 默认 image 大小为 1280x720。

示例:

```
SpmCameraDevice mDevice = SmartPlatformManager.get().openCameraDevice("0");  
mDevice.startPictureSequence(source, config);
```

1.2.2.7.30 stopPictureSequence

函数:

```
public void stopPictureSequence(@PictureSequenceSource int source)
```

细节:

停止连续获取图片, 和 startPictureSequence() 对应。

示例:

```
CameraDevice mDevice = SmartPlatformManager.get().openCameraDevice("0");  
mDevice.stopPictureSequence(source);
```

1.2.2.7.31 enableShareBuffer

函数:

```
public void enableShareBuffer(@PictureSequenceSource int source, boolean enable)
```

参数:

source: 获取连续图片的源

enable: 是否开启 FD 回调

细节:

此 API 通过 FD 连续获取图片数据。其中 YUV data 可以用于 ADAS 或者其它算法分析。

示例:

```
SpmCameraDevice mDevice = SmartPlatformManager.get().openCameraDevice("0");  
mDevice.startPictureSequence(source, config);  
mDevice.enableShareBuffer(source, true);
```

1.2.2.7.32 updateRequest

函数:

```
public void updateRequest()
```

细节:

当更新 SpmParameters 后需要调用此函数使其生效。

1.2.2.7.33 getCameraId

函数:

```
public String getCameraId()
```

细节:

获取当前 SpmCameraDevice 的 ID。

1.2.2.7.34 setAeStatusCallback

函数:

```
public void setAeStatusCallback(AeStatusCallback callback)
```

参数:

callback Ae 曝光值的回调

细节:

AeStatusCallback 定义如下:

```
public interface AeStatusCallback {
    void onStatusCallback (int value);
    void onStatusCallbackArray (int[] value,int size);
}
```

示例:

```
SpmCameraDevice mCameraDevice = SmartPlatformManager.get().openCameraDevice("0");
mCameraDevice.setAeStatusCallback (
    new AeStatusCallback () {
        public void onStatusCallback (int value){
            //TODO
        }
        public onStatusCallbackArray (int[] value,int size){
            //TODO
        }
    }
);
```

1.2.2.7.35 setDropStreamFrame

函数:

```
public int setDropStreamFrame(int streamType, int streamSource, int streamPolicy)
```

参数:

streamType YUV 0/record 1/preview 2
streamSource @PictureSequenceSource int source, @PreviewSource.Format int previewSource,
@RecordSource.Format int recordSource
streamPolicy 丢帧策略。

- 如果 param == 0, 不丢帧;
- 如果 param > 0, 每隔多少帧丢一帧;
- 如果 param < 0, 每隔一帧丢多少帧。

返回值:

返回一个整数值，表示设置单路丢帧是否成功，0 表示成功，否则失败。

示例:

```
mCameraDevice. setDropStreamFrame (1, RecordSource. GENERAL_CAMERA, 1);  
//preview 第 0 路 record 每隔 1 帧丢 1 帧
```

1.2.2.7.36 updateVideoGPSInfo

函数:

```
public void updateVideoGPSInfo(int latitude10000,int longitude10000, @RecordSource.Format  
int recordSource)
```

参数:

latitude10000 纬度*10000
longitude10000 经度 *10000
recordSource 录制的源

细节:

在录制过程中动态设置 GPS 位置信息，保存到 MP4 视频头文件信息中。

示例:

```
SpmCameraDevice mDevice = SmartPlatformManager.get().openCameraDevice("");  
mDevice. updateVideoGPSInfo (3140000,2240000,RecordSource.GENERAL_CAMERA);
```

1.2.2.8 SpmParameters

引入（导入） 参数:

```
import com.mediatek.smartplatform.SpmCameraDevice.SpmParameters;
```

实例化(创建)对象:

```
SpmCameraDevice mDevice = SmartPlatformManager.get().openCameraDevice("");  
SpmParameters params=mDevice.getParameters();
```

1.2.2.8.1 enableWatermarkImage

函数:

```
public void enableWatermarkImage(boolean enable)
```

参数:

enable 是否添加图片水印

细节:

参数为 true 表示在拍照图片中添加水印，false 表示不添加。该接口控制 preview，recording 和 capture 的图片水印。

示例:

```
SpmCameraDevice mDevice = SmartPlatformManager.get().openCameraDevice("");  
SpmParameters params=mDevice.getParameters();  
params.enableWatermarkImage(true);  
mDevice.setParameters(params);
```

1.2.2.8.2 setWatermarkImaArea

函数:

```
public void setWatermarkImgArea(int left, int top, int right, int bottom)
```

参数:

left 图片水印区域的 x 轴起始位置

top 图片水印区域的 y 轴起始位置

right 图片水印区域的 x 轴结束位置

bottom 图片水印区域的 y 轴结束位置

细节:

设置图片水印的区域，图片水印的宽度必须是 32 的整数倍，高度必须是 16 的整数倍，否则无法正常显示。图片的大小要和水印区域的大小一致。图片水印区域的起始位置和文字水印区域起始位置的计算方式一致，但是宽度和高度存在差别，通常将图片水印区域的宽度和高度设置成和图片大小一样即可。

示例:

```
params.setWatermarkImgArea(20,20,180,180);
```

1.2.2.8.3 setWatermarkImagePath

函数:

```
public void setWatermarkImgPath(String path)
```

参数:

path 水印图片的路径

细节:

设置水印图片的路径，必须是完整的路径。

示例:

```
params.setWatermarkArea("/sdcard/watermark.bmp")
```

1.2.2.8.4 enableWatermarkText

函数:

```
public void enablePictureWatermarkText(boolean enable)
```

参数:

enable 是否添加文字水印

细节:

参数为 true 表示添加水印，false 表示不添加。该接口控制 preview，recording 和 capture 等的文字水印接口。

示例:

```
SpmCameraDevice mDevice = SmartPlatformManager.get().openCameraDevice("0");
SpmParameters params=mDevice.getParameters();
params.enableWatermarkText(true);
mCameraDevice.setParameters(params);
```

1.2.2.8.5 setWatermarkTextArea

函数:

```
public void setWatermarkTextArea(int left, int top, int right, int bottom)
```

参数:

left 水印区域的 x 轴起始位置

top 水印区域的 y 轴起始位置

right 水印区域的 x 轴结束位置

bottom 水印区域的 y 轴结束位置

细节:

设置文字水印的区域，上述的四个参数对应的就是一个矩形区域，绘制的文字内容会在这个矩形区域内。如果文字内容超过了这个区域，会被截掉，不能显示出来。这个区域最终对应到画面的位置怎么计算：

起始位置 $x = \text{left} * \text{画面宽度} / 1000$;

起始位置 $y = \text{top} * \text{画面高度} / 1000$;

矩形区域宽度 $w = (\text{right} - \text{left}) * \text{画面宽度} / 1000$;

矩形区域高度 $h = (\text{bottom} - \text{top}) * \text{画面高度} / 1000$;

示例：

```
params.setWatermarkArea(20, 20, 60, 240);
```

1.2.2.8.6 setWatermarkText

函数：

```
public void setWatermarkText(String text)
```

参数：

text 水印显示的文本

细节：

设置水印显示的文字内容。

示例：

```
params.setWatermarkText("watermark");  
mCameraDevice.setParameters(params);
```

1.2.2.8.7 setWatermarkTextSize

函数：

```
public void setWatermarkTextSize(float size)
```

参数：

size 水印文字的大小

细节：

设置水印文字的大小，文字水印的大小是按照画面高度的比例计算的，计算公式：文字实际大小 = $\text{size} * \text{画面高度} / 1000$ ；单位是像素。所以文字水印的大小应与文字水印区域的高度相等，或略小于其高度，水印才能显示出来。

示例：

```
params.setWatermarkArea(20, 20, 60, 240); // 矩形宽度是 220，高度 40  
params.setWatermarkTextSize(40); // 文字水印大小应该设置成 40
```

1.2.2.8.8 setWatermarkTextColor

函数:

```
public void setWatermarkTextColor(int color)
```

参数:

color 水印文字的颜色，如果不设置，默认是红色。

细节:

设置水印的颜色，使用 ARGB 模式，可以通过设置 android.graphics.Color 对象来构造颜色。

示例:

```
params.setWatermarkTextColor(Color.RED); //显示为红色
```

1.2.2.8.9 setWatermarkTextPosition

函数:

```
public void setWatermarkTextPosition(float x,float y)
```

参数:

x 相对应水印区域起始位置的横向偏移量

y 相对应水印区域起始位置的竖向偏移量

细节:

绘制出来的文字水印可以理解成一个矩形图片，偏移量指的是文字水印左下角相对于文字水印区域起始点的偏移量。通常情况下，将 y 的值和文字水印区域的高度设置成一样即可。

示例:

```
params.setWatermarkArea(20,20,60,240); //矩形宽度是 220，高度 40
params.setWatermarkTextPosition(10,40); //y 值和矩形区域高度一致
```

1.2.2.8.10 setWatermarkFontFile

函数:

```
public void setWatermarkFontFile(String file)
```

参数:

file 水印的字体文档

1.2.2.8.11 setWatermarkTimeMs

函数:

```
public void setWatermarkTimeMs(boolean enable)
```

参数:

enable 是否开启时间水印

1.2.2.8.12 set

函数:

```
public void set(CaptureRequest.Key<T> key, T value)
```

参数:

设置摄像头的请求参数，可以参考原生 CaptureRequest/CaptureResult/CameraCharacteristics 中的 key 进行设定。

细节:

此方法适用于对 Android 支持的参数进行设置。

1.2.2.9 IpodProxy

引入 (导入) IpodProxy:

```
import com.mediatek.smartplatform.IpodProxy;
```

实例化(创建)对象:

```
IpodProxy proxy=SmartPlatformManager.get().getIpodProxy();
```

或者 IpodProxy proxy=IpodProxy.getInstance(SmartPlatformManager.get());

建议使用第一种方式获取 IpodProxy 对象，IpodProxy 也是采用单例模式。

1.2.2.9.1 doShutdown

函数:

```
public int doShutdown(String reason,boolean isShutdown)
```

参数:

reason 要关机的原因，目前可以设置为任意字符串。

isShutdown 是否要关机，true 表示关机，false 表示不关机。

返回值:

该函数的执行结果。

细节:

- 此 API 用于设定进入 IPO 后是否要真正关机。当前按 power 键关机，系统并没有真正关机，而是进入 IPO/IPOH 模式。当需要彻底关机时，可以调用此 API 实现该功能。在 IPOD 进程未启动之前，如果调用了此 API 且将传入的 isShutdown 参数设置为 true，还可以再次调用此 API，并将第二个参数 isShutdown 设置为 false 以取消 IPOD 关机。
- 需要注意的是参数 reason 不能包含有 “=” 或 “;” 这两个字符。
- 调用此 API 的返回值说明如下：
 - **RESULT_SUCCESS** 表示此 API 的参数已经成功传给 IPOD 进程去执行，这种情况一般发生在关机之后，并且 IPOD 进程已经运行。
 - **RESULT_PARAMS_SAVED** 表示此 API 的参数已经成功保存，等 IPOD 进程运行后会传给 IPOD 进程去执行。
 - **RESULT_PARAMS_REPLACED** 表示此 API 之前有调用过，此次传下来的参数覆盖之前的参数，也就是说最后一次传下来的参数会被传到 IPOD 那边执行。
 - **RESULT_FAIL** 表示调用此 API 调用失败了，失败的原因有很多，视具体情况而定。
 - **RESULT_EXCEPTION** 表示调用这个 API 的时候出现了异常，一般是 Smartplatform service 不存在了，也就是 SmartPlatformServer 进程死掉了。

示例:

```
IpodProxy proxy= SmartPlatformManager.get().getIpodProxy();
proxy.doShutdown("test",true);    //进入 ipo 后，进行关机操作
```

1.2.2.9.2 setRebootControl

函数:

```
public int setRebootControl(int flag)
```

参数:

flag 开启定时重启功能的标志，值为 0 或 1

返回值:

该函数的执行结果。

细节:

- 用于打开定时重启的功能，传入的 flag 参数为 1 时表示打开，参数为 0 时表示关闭。如果不调用 setRebootTimeSlot 去设定重启时间，则会以上次设定的时间作为重启时间。
- 调用此 API 的返回值说明如下：

- **RESULT_SUCCESS** 表示此 API 的参数已经成功传给 IPOD 进程去执行，这种情况一般发生在关机之后，并且 IPOD 进程已经运行。
- **RESULT_PARAMS_SAVED** 表示此 API 的参数已经成功保存，等 IPOD 进程运行后会传给 IPOD 进程去执行。
- **RESULT_PARAMS_REPLACED** 表示此 API 之前有调用过，此次传下来的参数覆盖之前的参数，也就是说最后一次传下来的参数会被传到 IPOD 那边执行。
- **RESULT_FAIL** 表示调用此 API 调用失败了，失败的原因有很多，视具体情况而定。
- **RESULT_EXCEPTION** 表示调用这个 API 的时候出现了异常，一般是 Carcorder Service 不存在了，也就是 SmartPlatformServer 进程已经死掉了。

示例：

```
IpodProxy proxy= SmartPlatformManager.get().getIpodProxy();  
proxy.setRebootControl(1);    //打开定时重启功能
```

1.2.2.9.3 setRebootTimeSlot

函数：

```
public int setRebootTimeSlot(int slotTime)
```

参数：

slotTime 重启的时间点，值的范围为 0 到 23

返回值：

该函数的执行结果。

细节：

- 此 API 用于设定重启的时间，参数是重启的时间点，范围在 0 到 23。要使当前的设置有效，需要调用 setRebootControl 去打开定时重启功能。
- 调用此 API 的返回值说明如下：
 - **RESULT_SUCCESS** 表示此 API 的参数已经成功传给 IPOD 进程去执行，这种情况一般发生在关机之后，并且 IPOD 进程已经运行起来了。
 - **RESULT_PARAMS_SAVED** 表示此 API 的参数已经成功保存，等 IPOD 进程运行起来后会传给 IPOD 进程去执行。
 - **RESULT_PARAMS_REPLACED** 表示此 API 之前有调用过，此次传下来的参数覆盖之前的参数，也就是说最后一次传下来的参数会被传到 IPOD 那边执行。
 - **RESULT_FAIL** 表示调用此 API 调用失败了，失败的原因有很多，视具体情况而定。
 - **RESULT_EXCEPTION** 表示调用这个 API 的时候出现了异常，一般是 Smartplatform Service 不存在了，也就是 SmartPlatformServer 进程已经死掉了。

示例:

```
IpodProxy proxy= SmartPlatformManager.get().getIpodProxy();  
proxy. setRebootControl(1);    //打开定时重启功能  
proxy. setRebootTimeSlot (5); //设置凌晨 5 点重启
```

1.2.2.9.4 exitIpod

函数:

```
public int exitIpod(int reason)
```

参数:

reason 退出 IPOD 的原因

返回值:

该函数的执行结果。

细节:

- 在 IPO 模式下, 调用这个 API 跟长按电源键开机是一样的功能, 需要在 IPOD 进程运行起来之后, 调用才有效。
- 调用此 API 的返回值说明如下:
 - RESULT_SUCCESS** 表示此 API 的参数已经成功传给 IPOD 进程去执行, 这种情况一般发生在关机之后, 并且 IPOD 进程已经运行起来了。
 - RESULT_PARAMS_SAVED** 表示此 API 的参数已经成功保存, 等 IPOD 进程运行起来后会传给 IPOD 进程去执行。
 - RESULT_PARAMS_REPLACED** 表示此 API 之前有调用过, 此次传下来的参数覆盖之前的参数, 也就是说最后一次传下来的参数会被传到 IPOD 那边执行。
 - RESULT_FAIL** 表示调用此 API 调用失败了, 失败的原因有很多, 视具体情况而定。
 - RESULT_EXCEPTION** 表示调用这个 API 的时候出现了异常, 一般是 Carcorder Service 不存在了, 也就是 SmartPlatformServer 进程已经死掉了。

示例:

```
IpodProxy proxy= SmartPlatformManager.get().getIpodProxy();  
proxy. exitIpod(0);    //退出 ipod, 然后开机
```

1.2.2.10 CarEventProxy

引入 (导入) CarEventProxy:

```
import com.mediatek.smartplatform.CarEventProxy;
```

实例化(创建)对象:

`CarEventProxy proxy=SmartPlatformManager.get().getCarEventProxy();`

或者 `CarEventProxy proxy= CarEventProxy.getInstance(SmartPlatformManager.get());`

建议使用第一种方式获取 CarEventProxy 对象，CarEventProxy 也是采用单例模式。

目前由于硬件限制，只有 ACC on/off 的行为，其它行为需根据硬件添加。

1.2.2.10.1 addEngineChangedCallback

函数:

```
public void addEngineChangedCallback(EngineChangedCallback callback)
```

参数:

callback 车子发动机状态改变时的回调，与 ACC on/off 时对应。

同一回调对象，注册多次，只能保存一个。

细节:

- EngineChangedCallback 定义如下

```
public interface EngineChangedCallback{
    public static final int CAR_ENGINE_FLAMEOUT =0;
    public static final int CAR_ENGINE_WORKING = 1;
    void onEngineChanged(int status);
}
```

- 车子发动机状态改变时会触发此回调，onEngineChanged 中的 status 表示发动机的状态。当该值为 CAR_ENGINE_FLAMEOUT 时，表示发动机处于熄火状态，当该值为 CAR_ENGINE_WORKING 时，表示发动机处于工作状态。
- 这两个事件 CAR_ENGINE_WORKING/CAR_ENGINE_FLAMEOUT 对应的就是 ACC on/off 事件。

示例:

```
CarEventProxy mCarEventProxy = SmartPlatformManager.get().getIpodProxy();
mCarEventProxy.addEngineChangedCallback(
    new EngineChangedCallback(){
        public void onEngineChanged (int status){
            //TODO
        }
    }
);
```

1.2.2.10.2 removeEngineChangedCallback

函数:

```
public void removeEngineChangedCallback(EngineChangedCallback callback)
```

参数:

callback 车子发动机状态改变时的回调

细节:

移除注册的回调对象。

示例:

```
CarEventProxy mCarEventProxy = SmartPlatformManager.get().getIpodProxy();  
mCarEventProxy.removeEngineChangedCallback(callback);
```

1.2.2.10.3 queryCarEngineState

函数:

```
public int queryCarEngineState()
```

返回值:

当前发动机工作的状态

细节:

目前定义有三种状态:

- CAR_STATE_UNKNOWN 状态未知, 一般是调用 API 发生异常才会出现这种情况;
- CAR_ENGINE_FLAMEOUT, 表示发动机处于熄火状态;
- CAR_ENGINE_WORKING, 表示发动机处于工作状态。

直接将 Smartplatform Service 保存的发动机状态直接返回。

示例:

```
CarEventProxy mCarEventProxy = SmartPlatformManager.get().getIpodProxy();  
int state= mCarEventProxy.queryCarEngineState();
```

1.2.2.10.4 setDefaultAccOffBehavior

函数:

```
public void setDefaultAccOffBehavior(boolean enable)
```

参数:

enable 是否启用 ACC 的默认行为

细节:

- 默认的 ACC 行为是底层的驱动程序检测到 ACC 点火或者熄火时, 会将这一事件通过广播的形式发送给应用程序, 应用程序只需要注册对应的广播就可以了。
 - 点火事件广播: android.intent.action.ACTION_POWER_CONNECTED

- 熄火事件广播: android.intent.action.ACTION_POWER_DISCONNECTED
- 如果应用程序想通过回调的方式接收 ACC 点火或者熄火的事件，将参数值设为 false，同时通过 addEngineChangedCallback 来注册对应的回调。

示例:

```
CarEventProxy mCarEventProxy = SmartPlatformManager.get().getIpodProxy();
mCarEventProxy.addEngineChangedCallback(
    new EngineChangedCallback(){
        public void onEngineChanged (int status){
            //TODO
        }
    }
);
mCarEventProxy.setDefaultAccOffBehavior(false);
```

1.2.2.10.5 addCarReverseCallback

函数:

```
public void addCarReverseCallback(CarReverseCallback callback)
```

参数:

callback 车子运动方向改变时的回调。
同一回调对象，注册多次，只能保存一个。

细节:

- CarReverseCallback 定义如下:

```
public interface CarReverseCallback {
    public static final int CAR_STATUS_NORMAL = 0;
    public static final int CAR_STATUS_LEFT = 1;
    public static final int CAT_STATUS_RIGHT = 2;
    public static final int CAR_STATUS_REVERSE = 3;

    public static final int CAR_STATUS_SOURCE_AVM = 1;
    public static final int CAR_STATUS_SOURCE_GPIO = 2;

    void onReverse(int status, int source);
}
```

- 当车子从左转/右转/倒车/前行中某一状态转换成另外不同的状态时触发此回调。触发此回调的来源有两个，GPIO 或者 AVM。两种触发方式都需要有硬件的支持。

示例:

```
CarEventProxy mCarEventProxy = SmartPlatformManager.get().getIpodProxy();
mCarEventProxy.addCarReverseCallback(
    new CarReverseCallback(){
        public void onReverse(int status, int source){
            //TODO
        }
    }
);
```

```
    }  
);
```

1.2.2.10.6 removeCarReverseCallback

函数:

```
public void removeCarReverseCallback(CarReverseCallback callback)
```

参数:

callback 车子运动状态发生变化时的回调

细节:

移除注册的回调对象。

示例:

```
CarEventProxy mCarEventProxy = SmartPlatformManager.get().getIpodProxy();  
mCarEventProxy.removeCarReverseCallback(callback);
```

1.2.2.11 CollisionProxy

引入 (导入) CollisionProxy:

```
import com.mediatek.smartplatform.CollisionProxy;
```

实例化(创建)对象:

```
CollisionProxy proxy=SmartPlatformManager.get().getCollisionProxy();
```

或者 CollisionProxy proxy= CollisionProxy.getInstance(SmartPlatformManager.get());

建议使用第一种方式获取 CollisionProxy 对象，CollisionProxy 也是采用单例模式。

1.2.2.11.1 addCollisionCallback

函数:

```
public void addCollisionCallback(CollisionCallback callback)
```

参数:

callback 平台检测到碰撞时的回调

细节:

- CollisionCallback 定义如下:

```
public interface CollisionCallback{  
    public static final int COLLISION_UNRELIABLE =0;  
    public static final int COLLISION_LOW=1;  
    public static final int COLLISION_MEDIUM=2;  
    public static final int COLLISION_HIGH=3;
```

```
void onCollision(int collision,int status);  
}
```

- 平台检测到碰撞时会触发此回调，onCollision 中的 collision 实际指的是碰撞的等级（灵敏度），目前分为三个等级 COLLISION_LOW，COLLISION_MEDIUM，COLLISION_HIGH。status 为保留参数，一般为 0，如果修改了碰撞检测算法，则该值和 collision_detect()中的 (*result) 值相等。
- 同一个对象，只能注册一次，即使该对象被添加了多次，但最终只会保存一个回调对象，同样只需要调用一次 removeCollisionCallback 就可以将该回调彻底移除。

示例：

```
SmartPlatformManager mSmartPlatformManager = SmartPlatformManager.get();  
mSmartPlatformManager.addCollisionCallback(  
    new CollisionCallback() {  
        public void onCollision(int collision,int status){  
            //TODO  
        }  
    }  
);
```

1.2.2.11.2 removeCollisionCallback

函数：

```
public void removeCollisionCallback (CollisionCallback callback)
```

参数：

callback 平台检测到碰撞时的回调

细节：

移除注册的回调对象。值得注意的是，如果同一个回调对象，注册了多次，只要调用该函数一次就能完全移除。

示例：

```
SmartPlatformManager mSmartPlatformManager = SmartPlatformManager.get();  
mSmartPlatformManager.removeCollisionCallback(callback);
```

1.2.2.11.3 setSuspendCollision

函数：

```
public void setSuspendCollision(boolean fgEnabled)
```

参数：

fgEnabled 是否打开 Gsensor 检测碰撞的功能

细节：

- 通过这个 API 可以实现打开或关闭 Gsensor 检测碰撞的功能。参数为 true 表示打开，false 表示关闭。这个 API 对应的功能是碰撞开机。默认情况下这个功能是打开的，可以通过 getSuspendCollision 查看是否有开启这个功能。
- 正常开机还是碰撞开机，可以通过开机时发出来的广播区分。

示例：

```
SmartPlatformManager mSmartPlatformManager = SmartPlatformManager.get();  
//打开 gsensor 检测碰撞的功能，即使在 ipo 状态下，如果发生了碰撞就会开机  
mSmartPlatformManager.setSuspendCollision(true);
```

1.2.2.11.4 getSuspendCollision

函数：

```
public boolean getSuspendCollision()
```

返回值：

返回一个 boolean 值，表示是否打开 Gsensor Driver 检测碰撞。

细节：

如果返回值为 true 表示打开，返回值为 false 表示关闭。

示例：

```
SmartPlatformManager mSmartPlatformManager = SmartPlatformManager.get();  
boolean isEnabled=mSmartPlatformManager.getSuspendCollision();
```

1.2.2.11.5 setNormalCollision

函数：

```
public void setNormalCollision(boolean isEnabled)
```

参数：

isEnabled 是否打开 Carcorder 检测碰撞的功能

细节：

Carcorder 检测碰撞可以理解为 Carcorder 进程里面的一个功能模块，通过此接口可以控制这个功能的开关。只有打开了这个功能，在发生碰撞时，才能在 CollisionCallback 收到回调。Carcorder 检测碰撞和 Gsensor 检测碰撞属于不同的模块，关闭了 Gsensor 检测碰撞的功能，并不会影响到 Carcorder 检测碰撞的功能，同样关闭了 Carcorder 检测碰撞的功能，也不会影响 Gsensor 检测碰撞的功能，所以这两个功能互不影响。默认情况下这个功能是打开的，如果上一次有设置过，则继续使用上一次的设置。

示例：

```
SmartPlatformManager mSmartPlatformManager = SmartPlatformManager.get();
```

```
mSmartPlatformManager.addCollisionCallback(  
    new CollisionCallback {  
        public void onCollision(int collision,int status){  
        }  
    }  
);  
mSmartPlatformManager.setNormalCollision(true);
```

1.2.2.11.6 getNormalCollision

函数:

```
public boolean getNormalCollision()
```

返回值:

返回一个 boolean 值，表示是否启用了 Carcorder 检测碰撞功能。

细节:

如果返回值为 true 表示打开，返回值为 false 表示关闭。

示例:

```
SmartPlatformManager mSmartPlatformManager = SmartPlatformManager.get();  
boolean isEnabled=mSmartPlatformManager.getNormalCollision();
```

1.2.2.11.7 setSuspendCollisionSensity

函数:

```
public void setSuspendCollisionSensity(int level)
```

参数:

level 碰撞的灵敏度

细节:

参数 level 代表灵敏度，也就是说当 Gsensor 的变化值超过某个阈值时，就认为发生了碰撞，这个阈值和 level 是相对应的。目前定义了 COLLISION_LOW，COLLISION_MEDIUM，COLLISION_HIGH 三个等级。在不设置的情况下，默认是 COLLISION_MEDIUM。

示例:

```
SmartPlatformManager mSmartPlatformManager = SmartPlatformManager.get();  
mSmartPlatformManager.setSuspendCollisionSensity(CollisionCallback.COLLISION_HIGH);
```

1.2.2.11.8 getSuspendCollisionSensity

函数:

```
public int getSuspendCollisionSensity()
```

返回值:

返回一个整数值，表示驱动程序会检测到碰撞的灵敏度。

细节:

目前定义了 `COLLISION_LOW`，`COLLISION_MEDIUM`，`COLLISION_HIGH` 三个等级。`COLLISION_HIGH` 最容易检测到碰撞，直观表现为设置成 `COLLISION_HIGH`，可能轻轻碰撞平台，就会检测到碰撞。

示例:

```
SmartPlatformManager mSmartPlatformManager = SmartPlatformManager.get();  
int level=mSmartPlatformManager.getDriverCollisionSensity();
```

1.2.2.11.9 setNormalCollisionSensity

函数:

```
public void setNormalCollisionSensity(int level)
```

参数:

level 碰撞的灵敏度

细节:

参数 level 代表灵敏度，也就是说 Gsensor 的变化值超过某个阈值时，就认为发生了碰撞，这个阈值和 level 是相对应的，目前定义了 `COLLISION_LOW`，`COLLISION_MEDIUM`，`COLLISION_HIGH` 三个等级。虽然这个函数和 `setSuspendCollisionSensity()` 很相似，但是它们是属于不同模块，互不影响。如果需要自定义碰撞的阈值，可以通过 `setCollisionThreshold` 来完成。`setCollisionThreshold()` 可以完全取代该接口，因为设置的灵敏度，到下面会转为对应的阈值，这和直接修改阈值的效果是一样的。

示例:

```
SmartPlatformManager mSmartPlatformManager = SmartPlatformManager.get();  
mSmartPlatformManager.setNormalCollisionSensity(CollisionCallback.COLLISION_HIGH);
```

1.2.2.11.10 getNormalCollisionSensity

函数:

```
public int getNormalCollisionSensity()
```

返回值:

返回一个整数值，表示 Carcorder 会检测到碰撞的灵敏度。

细节:

目前定义了 `COLLISION_LOW` , `COLLISION_MEDIUM` , `COLLISION_HIGH` 三个等级。`COLLISION_HIGH` 最容易检测到碰撞。直观表现为设置成 `COLLISION_HIGH` , 可能轻轻碰撞平台, 就会检测到碰撞。如果使用 `setCollisionThreshold` 来设置定义的碰撞阈值, 那么此函数得到的返回值就是 `setCollisionThreshold` 设定的 `level` 值。

示例:

```
SmartPlatformManager mSmartPlatformManager = SmartPlatformManager.get();  
int level=mSmartPlatformManager.getNormalCollisionSensity();
```

1.2.2.11.11 setCollisionThreshold

函数:

```
public int setCollisionThreshold(float x,float y,float z,int level)
```

参数:

x Gsensor X 轴方向加速度的阈值

y Gsensor Y 轴方向加速度的阈值

z Gsensor Z 轴方向加速度的阈值

level 设置为此阈值代表的等级

返回值:

返回一个整数值, 表示参数设置是否成功, 0 表示成功, 否则失败。

细节:

- 通过这个 API 可以自定义碰撞的阈值, 分别是 Gsensor X, Y, Z 轴方向加速度的阈值。之前的碰撞检测算法是只要 Gsensor 某个方向的加速度值超过了对应的阈值, 就认为是检测到碰撞了, 现在的算法是如果 Gsensor 某个方向的加速度变化超过设置的阈值, 就认为检测到碰撞。
- 第四个参数值 level 没有任何限定, 可以是任意整数, 这个参数的作用类似一个标识, 当检测到碰撞后, `CollisionCallback.onCollision()` 中第一个参数就是设置的这个值。通过 `getNormalCollisionSensity` 也能得到该值。
- 该函数和 `setNormalCollisionSensity()` 的作用是一样的, 所以不需要两个接口都调用。如果两个接口都调用了, 下面只会使用最后一个接口设置下去的参数, 建议使用该接口。

示例:

```
SmartPlatformManager mSmartPlatformManager = SmartPlatformManager.get();  
mSmartPlatformManager.setCollisionThreshold(5f,5f,5f,2);
```

1.2.2.11.12 getCollisionThreshold

函数:

```
public float[] getCollisionThreshold()
```

返回值:

返回设置的 Gsensor X, Y, Z 轴方向加速度的阈值

细节:

通过此 API 可以获取 setCollisionThreshold 设置的 Gsensor X, Y, Z 轴方向加速度的阈值。如果返回值为 null, 表示在获取阈值的时候出现了异常。

示例:

```
SmartPlatformManager mSmartPlatformManager = SmartPlatformManager.get();  
float[] f=mSmartPlatformManager.getCollisionThreshold();
```

1.2.2.11.13 setGsensorEventRate

函数:

```
public int setGsensorEventRate(int delayMs)
```

参数:

delayMs Gsensor 数据上报时间间隔

返回值:

返回一个整数值, 表示参数设置是否成功, 0 表示成功, 否则失败。

细节:

- 通过这个 API 可以设置 Gsensor 数据上报的时间间隔, 单位是毫秒。例如设置为 10, 意味着每隔 10 毫秒会收到一笔 Gsensor 数据, 换算成频率就是 100Hz。
- 该接口只是调用 sensorservice 的设置接口, 再由 sensorservice 设置到驱动程序端。传进去的值和实际设置到驱动程序的值不一定相等, 因为这个值有范围限定。

示例:

```
SmartPlatformManager mSmartPlatformManager = SmartPlatformManager.get();  
mSmartPlatformManager.setGsensorEventRate(10);
```

1.2.2.11.14 getGsensorEventRate

函数:

```
public int getGsensorEventRate()
```

返回值:

返回设置下去的 Gsensor 数据上报时间间隔

细节:

该接口只是拿到上一次调用 setGsensorEventRate()传下来的值，而不是拿到驱动程序那边实际使用的值，目前没有办法拿到驱动程序使用的值。

示例：

```
SmartPlatformManager mSmartPlatformManager = SmartPlatformManager.get();
int rate=mSmartPlatformManager.getGsensorEventRate();
```

1.2.2.11.15 setIPOStandby

函数：

```
public void setIPOStandby(Boolean on)
```

参数：

On true 表示进入 IPO 待机状态，false 表示退出 IPO 待机状态。

细节：

NATIVE Collision 根据是否进入 IPO 待机状态，选择不同流程从而达到省电的目的。

示例：

```
SmartPlatformManager mSmartPlatformManager = SmartPlatformManager.get();
mSmartPlatformManager.setIPOStandby ();
```

1.3 配置/客制化指南

Codebase 路径：vendor\mediatek\proprietary\frameworks\base\smartplatform\sdk

APK demo 路径：vendor\mediatek\proprietary\packages\apps\CarcorderMcamDemo

1.3.1 主要功能介绍

1.3.1.1 预览

开启预览：

```
SpmCameraDevice mDevice = SmartPlatformManager.get().openCameraDevice("0");
SurfaceView surfaceView=findViewById(R.id.surface_view);
SurfaceHolder holder=surfaceView.getHolder();
mDevice.setPreviewSurface(holder.getSurface(),PreviewSource.GENERAL_CAMERA);
mDevice.starPreview();
```

停止预览：

```
SpmCameraDevice mDevice = SmartPlatformManager.get().openCameraDevice("0");
```

```
mDevice.stopPreview();  
mDevice.setPreviewSurface(null, PreviewSource.GENERAL_CAMERA);
```

1.3.1.2 拍照

```
SpmCameraDevice mCameraDevice = SmartPlatformManager.get().openCameraDevice("0");  
String fileName="/sdcard/DCIM/photo/"+filename;  
mCameraDevice.takePicture(fileName,shutter,jpeg);
```

1.3.1.3 录制

开启录制:

```
RecordConfiguration recordConfig= RecordConfiguration.get(recordSource);  
recordConfig.setVideoCallback(mVideoCallback);  
mCameraDevice.startRecord(RecordSource.GENERAL_CAMERA, recordConfig);
```

停止录制:

```
SpmCameraDevice Device = SmartPlatformManager.get().openCameraDevice("0");  
mDevice.stopRecord();
```

注: 录制涉及到各种特性, 例如子码流, 主码流, 打点保护等, 在 DVR 文档中会详细介绍。

1.3.1.4 YUV Callback

```
SpmCameraDevice.ImageDataCallback mSequenceCallback = new SpmCameraDevice.ImageDataCallback  
(() {  
    Public void onImageAvailable(ImageDataCallbackInfo imageDataCallbackInfo){  
    }  
})  
  
SpmCameraDevice mDevice = SmartPlatformManager.get().openCameraDevice("0");  
PictureConfiguration picConfig = PictureConfiguration.get(PictureSequenceSource.  
GENERAL_CAMERA);  
picConfig.mImageDataCallback = mSequenceCallback;  
mDevice.startPictureSequence(source ,picConfig);
```

注: imageDataCallback 返回的是 data, 如果需要 FD 回调, 需要在 startPictureSequence 通过 enableShareBuffer 获取 FD。

1.3.2 碰撞检测特别说明

1.3.2.1 碰撞启动某个录制 APK

需求说明:

在开机状态下，当 Demo APK 退出后，通过注册回调的方式是无法收到 Carcorder Service 发送的碰撞通知。现在想要在 APK 退出后（没有运行）的情况下也能收到碰撞事件，目前是通过广播的形式将碰撞事件通知 APK。

实例代码:

1. 实现一个继承 BroadcastReceiver 的类。

```
class CollisionBroadcastReceiver extends BroadcastReceiver{
@Override
publicvoid onReceive(Context context, Intent intent)
{
}
}
```

2. 应该使用静态注册的方式，在 Manifest.xml 里面注册广播。

```
<receiverandroid:name=".CollisionBroadcastReceiver ">
<intent-filter>
<action
android:name="android.intent.action.GSENSOR_COLLISION"/>
</intent-filter>
</receiver>
```

1.3.2.2 碰撞检测算法开放与客制化

代码路径:

alps/frameworks/native/libs/smartplatform/libsmartplatformcollision/*

文件组成:

SmartPlatformCollision.h SmartplatformCollision.cpp Android.mk

接口定义:

下面两个函数的定义不能修改，函数名、参数和返回值都不能修改。

```
void collision_init();
```

初始化函数：当 collide 运行起来，libsmartplatformcollision.so 加载完成后会调用一次此函数。仅会调用一次，用于变量的初始化。

```
void collision_detect(AsensorEvent value, const float* threshod, int size, int level, int* result);
```

参数说明:

AsensorEvent value 封装传感器数据的结构体。

`const float* threshod x, y, z` 阈值数组, 和 `setCollisionThreshold()` 设置的值一致。

`int size` 指定 `threshod` 数组的长度, 目前定义的长度是 3。

`int level` 碰撞的等级, 也是设置过的值。

`int* result` 额外的返回值, 应用程序可以通过 `CollisionCallback.onCollision(int collision, int status)` 获取, `collision` 和 `level` 的值相等, `status` 和 `*result` 的值相等。

返回值说明:

如果返回 `true`, 则认为检测到碰撞, 会通过回调的方式通知应用程序, 如果应用程序不注册回调, 则会发出广播。

如果返回 `false`, 则认为没有检测到碰撞, 既不会回调, 也不会发广播。

其他说明:

`sensorservice` 每送一次数据过来, 就会调用一次 `collision_detect` 这个函数。

可以通过 `setNormalCollision()` 设置是否启用这个功能。如果禁用了这个功能, 即使有传感器数据传送过来, 也不会回调 `collision_detect`。

参考实例:

请参考 `SmartplatformCollision.h`, `SmartPlatformCollision.cpp` 的实现方式。

1.4 常见问题/故障排除

1.4.1 SDK 常见问题

1.4.1.1 调用 `setpreviewSurface` 顺序不对导致底层缓冲区卡住

如果先调用 `setpreviewSurface null` 再去调用 `stop preview`, 会打印出如下日志:

```
BufferQueueProducer: [ImageReader-1280x720f32315659m5-15749-52] (this:0x7a989dd000, id:52, api:4, p:544, c:-1) queueBuffer: BufferQueue has been abandoned
```

如果还未调用 `stop preview` 就销毁了 `surface`, 会导致底层缓冲区卡住并报错。

1.4.1.2 拍照 `takePicture` 另开线程

有的 SD Card 保存图片慢, 为了避免阻塞 APK, 在调用 `takePicture` 时可另开线程来执行保存操作。

```
new Thread(() -> {  
    mDevice.takePicture(path, mShutterCallback, pictureCallback);  
}).start();
```

1.4.1.3 不能在回调中做耗时任务

考虑到同步和效率问题，SDK 中的回调是直接 `在 binder 线程中处理`，并未新开线程处理，所以应用程序在收到回调之后，最好是新开线程来处理；如果未新开线程处理，不能继续在回调中调用 SDK 接口（可能造成死锁）；也不要再在回调中做耗时较久的事情，这样容易造成 APK ANR。

1.4.2 调试日志开关方法

1.4.2.1 打开 SpmCameraDeviceImpl 调试日志的方法

```
adb shell setprop persist.vendor.log.spm sdk true;
adb reboot
```

1.4.2.2 关闭提前设定 Capture Surface 的方法

```
adb shell setprop persist.vendor.spm.presetpic false
adb reboot
```

附件一 附加条款

您(或您所代表的公司或其他法律主体, 以下合称「您」)是否得取得、下载与使用本文件及其相关信息皆应以您接受本附加条款为先决要件。您对于本文件之使用、取得或下载即表示您已接受本附加条款并同意受本附加条款之拘束。若您不同意受本附加条款之拘束, 您将不得使用、取得或下载本文件并应立即删除或毁弃所有本文件之副本。

本文件含有联发科技股份有限公司及其关联公司(以下合称「联发科技」)或其授权人之机密信息及专有信息, 仅供您为本文件所描述之联发科技芯片组作内部使用而不得用于其他任何目的(包括但不限于确认或提供支持任何对联发科技、其供货商及/或其直接或间接客户所提潜在专利侵权主张的证据)。禁止未经授权使用或揭露本文件及其中所含信息。您同意赔偿联发科技因您未经授权使用或揭露本文件及其中所含信息之一部或全部导致联发科技所受之任何损失或损害。

联发科技及其授权人保有本文件的所有权及组成所有权之各项权利且未针对任何知识产权授权(无论明示或默示、透过禁反言原则或其他方式)。联发科技得随时变更本文件内容而无须另行通知。联发科技无须承担与使用或信赖本文件相关或因使用或信赖本文件致生之任何责任, 包括但不限于间接损害或附带损害赔偿任。

本文件及联发科技所提供关于本文件之任何其他材料、信息或技术支持, 均以现况交付为准, 联发科技不负任何明示、默示、法定或其他形式之担保责任, 特别是适销性、不侵权、针对特定用途之适用性、完整性或正确性之担保责任或任何由贸易惯例或交易、履行过程所生之担保责任。对于联发科技为符合您所提规格或遵循特定标准或标准组织的要求所作之交付物, 联发科技亦不应承担任何责任。

于未对前述条款造成限制之情形下, 联发科技不对其产品任何特定用途之适用性承担任何保证或担保责任, 亦不承担任何因产品、电路或软件之应用或使用致生之赔偿责任。您同意您应单独承担关于整合您的产品与联发科技产品所涉及之设计、验证与测试之所有责任, 并确保前述整合产品符合相关标准及任何安全性要求或其他要件。

本附加条款及所有与本附加条款或本文件相关之行为应以台湾法律管辖、解释与阐明, 不适用冲突法原则。